

Calculus, optimization & *information theory*.

Measure change. Improve a prediction.

Connect gradients, probability, and the objectives behind learning.

🕒 25-30 min core reading · Build on Chapters 6-7 · 12 short sections

Calculus describes change. Optimization uses that information to adjust parameters.

Information theory helps define what a good probability prediction means. Together, they explain much of the mathematics behind model training.

WHAT YOU WILL BE ABLE TO DO

Read a derivative, follow the chain rule, calculate a gradient update, and explain why a learning rate can fail. Distinguish entropy, cross-entropy, and KL divergence, then connect a probability loss to its gradient.

Reading path: Begin with one curve, follow a small calculation graph, and experiment with gradient descent. Then connect Chapter 7's probabilities to learning objectives. Core examples use small numbers; proofs and advanced connections stay in optional notes.

1. Measure how fast a function changes

For $f(x) = x^2$, increasing x from 3 to 4 changes the output from 9 to 16. The **average rate of change** is $(16 - 9) / (4 - 3) = 7$. This is the slope of the line joining the two points, called a **secant**.

For a smaller change from 3 to 3.1, the slope is $(9.61 - 9) / 0.1 = 6.1$. Shrink the change further and this slope approaches 6. The **derivative** $f'(3) = 6$ describes the instantaneous rate at $x = 3$: the slope of the **tangent** there.

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} \quad \text{so for } f(x) = x^2, \quad f'(x) = 2x$$

A **limit** describes what a quantity approaches. We approach $h = 0$; we do not substitute zero into a division by h . For x^2 , expanding the numerator gives $2xh + h^2$, so the quotient is $2x + h$, approaching $2x$.

➤ A tangent describes a local rate

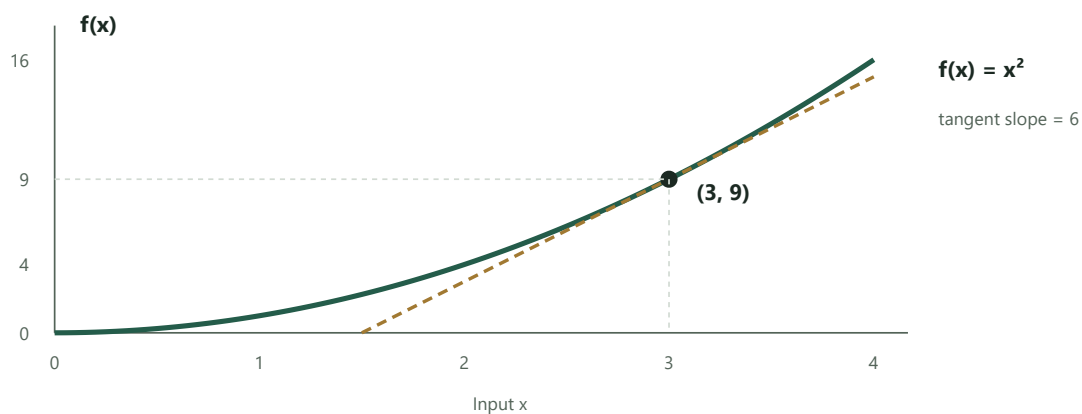


Figure 1. Dimensionless $f(x) = x^2$ (solid) and its tangent at $x = 3$ (dashed). The tangent has slope 6 and agrees with the curve at $(3, 9)$; it is only a local approximation.

A positive derivative means increasing x slightly increases f locally; a negative derivative means it decreases f . A zero derivative means zero first-order slope, not necessarily a minimum. For a small change Δx , the local approximation is $\Delta f \approx f'(x)\Delta x$. At $x = 3$, $\Delta x = 0.1$ predicts a change of 0.6; the exact change is 0.61. The approximation need not work for a large move.

Derivative units are output units divided by input units. When differentiating loss with respect to a weight, the question is how loss changes as that weight changes-not how the input itself changes. D2L's calculus introduction provides further derivations.

▼ Go deeper: derivative rules and points without a derivative

For a constant c , $d(c)/dx = 0$. For a positive integer n , $d(x^n)/dx = nx^{n-1}$. Derivatives distribute over sums and constant scaling: $d(3x^2 + 2x + 7)/dx = 6x + 2$. Two other useful rules are $d(e^x)/dx = e^x$ and $d(\ln x)/dx = 1/x$ for $x > 0$.

The **product rule** is $(uv)' = u'v + uv'$, not $u'v'$. The **quotient rule** is $(u/v)' = (u'v - uv')/v^2$ where v is nonzero. The chain rule, introduced below, handles one function inside another.

A function is **continuous** at a point when its nearby values approach its value there. Differentiability implies continuity, but continuity does not imply differentiability. The absolute-value function $|x|$ is continuous at zero but has slopes -1 and $+1$ on its two sides, so no ordinary derivative there. ReLU, $\max(0, x)$, also has a kink at zero. Optimization libraries use specified conventions at such points; later neural-network chapters explain how these functions are used.

2. Move from one derivative to a gradient

A model usually has many parameters. A **partial derivative** changes one variable while holding the others fixed. A **gradient** collects those partial derivatives into a vector, in a stated order.

Use one dimensionless training example: input $x = 2$, target $y = 5$, prediction $\hat{y} = wx + b$. With $w = 1$ and $b = 1$, the prediction is 3. The **residual** $r = \hat{y} - y$ is -2 , and the half-squared loss is $L = r^2/2 = 2$. The factor $1/2$ simplifies the derivative without changing where the minimum occurs.

$$\frac{\partial L}{\partial w} = (wx + b - y)x = -4, \quad \frac{\partial L}{\partial b} = wx + b - y = -2$$

Thus $\nabla L = [-4, -2]$ in $[w, b]$ order. The symbol ∂ marks a partial derivative; ∇ is read "gradient" or "nabla." These numbers describe two different sensitivities at the current parameters. They are not replacement parameter values.

For a differentiable scalar function, the gradient points toward steepest local increase measured using ordinary Euclidean distance in the chosen coordinates. Its negative points toward steepest local decrease. This is local information; the shape farther away and the step length still matter.

▼ Go deeper: Jacobians and Hessians connect to Chapter 6

A **Jacobian** is a matrix of first partial derivatives when a function has multiple outputs. Its row i and column j describe how output i changes with input j . For a scalar loss, the gradient is commonly written as a column vector; the Jacobian of that scalar output is its transpose under this convention.

A **Hessian** collects second partial derivatives of a scalar function into a matrix. It describes local curvature. For $L(w) = (w - 3)^2/2$, the second derivative is 1 everywhere. For a smooth multivariable function, positive-definite curvature at a stationary point implies a strict local minimum; mixed positive and negative curvature indicates a saddle. A zero or singular Hessian can require further analysis.

3. Follow the chain rule through a calculation

The **chain rule** connects local rates along a composition. If a change in w changes a prediction, which changes a residual, which changes a loss, multiply the sensitivities along that path.

Hold $x = 2$, $b = 1$, and $y = 5$ fixed. Define $p = 2w + 1$, $r = p - 5$, and $L = r^2/2$. At $w = 1$, the forward calculation gives $p = 3$, $r = -2$, and $L = 2$.

$$\frac{dL}{dw} = \frac{dL}{dr} \frac{dr}{dp} \frac{dp}{dw} = (-2)(1)(2) = -4$$

➤ Forward values, backward sensitivities

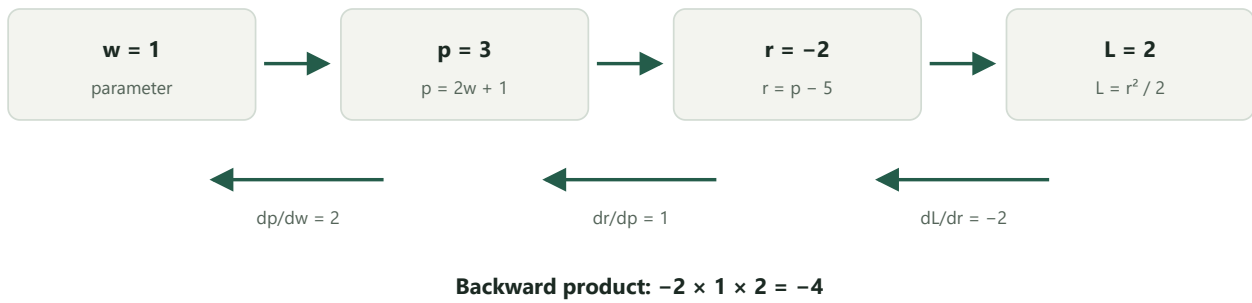


Figure 2. At $w = 1$, follow the values left to right and multiply the local derivatives right to left: $dL/dw = -2 \times 1 \times 2 = -4$. Input $x = 2$, bias $b = 1$, and target $y = 5$ are held fixed.

The loss changes by approximately $-4\Delta w$ for a small change to w at this point. If a parameter affects a loss through several paths, add the contributions from all those paths.

Automatic differentiation evaluates derivatives by applying differentiation rules to the operations in a computation. **Backpropagation** is the reverse-mode application used to efficiently obtain gradients of a scalar loss with respect to many parameters. The forward pass calculates values; the backward pass propagates sensitivities. An **optimizer** then uses those gradients to update parameters. Computing gradients alone does not change the parameters. See D2L's [automatic differentiation explanation](#).

▼ Go deeper: numerical differentiation and a gradient check

A **finite difference** estimates a derivative by evaluating nearby inputs. A central difference uses $[f(w + h) - f(w - h)] / (2h)$. It is useful for checking an implemented gradient on a small, smooth example, but requires extra function evaluations and can suffer from approximation and floating-point rounding errors. An arbitrarily tiny h is not always better.

Automatic differentiation applies the chain rule to executed operations, with ordinary floating-point precision; it does not estimate every derivative by perturbing every weight. **Symbolic differentiation** instead manipulates expressions to produce derivative formulas. These are three different approaches.

Detaching a value from a framework's calculation graph prevents gradients from following that connection. Whether a gradient should flow is part of the model design; it is not fixed by changing the learning rate. We will use framework autograd directly in the neural-network chapters.

4. Take a controlled step downhill

Gradient descent subtracts the gradient times a positive **learning rate** η ("eta"). For a parameter vector θ ("theta"), all gradient components are evaluated at the current parameters before updating:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

The subscript t counts updates. The learning rate controls the size of a move; it is a chosen **hyperparameter**, distinct from the fitted parameters θ .

For the experiment below, switch to the simpler, dimensionless objective $L(w) = (w - 3)^2/2$, whose derivative is $w - 3$. This is a separate teaching function, not the previous training example or Chapter 3's runtime dataset. Starting at $w = 0$ with $\eta = 0.5$:

gradient = -3 → **new w = 0 - 0.5(-3) = 1.5** → **new loss = 1.125**, down from 4.5.

ONE UPDATE AT A TIME

Live gradient descent

Same curve. Different step sizes.

Minimize $L(w) = (w - 3)^2/2$ with derivative $w - 3$. These are real updates on a dimensionless teaching function, not a trained application model.

Start at $w = 0$, where loss = 4.5. After five updates:

Learning rate	Weight	Loss	Behavior from this start
0.1	1.22853	1.569053	Approaches slowly
0.5	2.90625	0.004395	Approaches minimum
1	3	0	Reaches in one step
1.5	3.09375	0.004395	Alternates and approaches
2	6	4.5	Cycles, same loss
2.2	10.46496	27.862814	Alternates and moves away

At $\eta = 0.5$, the first update gives $w = 1.5$ and loss = 1.125. These are exact update rules with rounded displayed results. Starting at $w = 3$ leaves every displayed rate at the minimum.

Try $\eta = 0.5$, then 1.5, then 2 and 2.2, starting at zero each time. Crossing the minimum does not automatically mean failure: at 1.5 the iterates alternate sides but get closer. At 2 they cycle; at 2.2 they move farther away. The [gradient-descent chapter in D2L](#) develops the role of the step size.

▼ Go deeper: why these particular learning rates behave differently

For this quadratic, subtracting the optimum 3 from both sides of the update gives $w_{t+1} - 3 = (1 - \eta)(w_t - 3)$. Each update multiplies the error by $1 - \eta$. Starting away from the minimum, convergence occurs when $|1 - \eta|$ is less than 1: $0 < \eta < 2$. At $\eta = 1$, the error becomes zero in one step. At $\eta = 2$, it changes sign but keeps its magnitude.

This range belongs to this objective's curvature; it is not a universal safe range for neural networks. Multiplying the loss by 100 multiplies the gradient by 100, changing suitable step sizes. If you start exactly at $w = 3$, the gradient is zero and every displayed rate leaves it there.

5. Understand what an optimizer can and cannot solve

A **global minimum** is best over the allowed domain; a **local minimum** is best within a neighborhood. A **stationary point** has zero gradient, but might be a minimum, maximum, or saddle. For $f(x) = -x^2$, the zero slope at zero is a maximum. For $f(x, y) = x^2 - y^2$, the origin is a saddle: moving one way increases the value and another decreases it.

A **convex** function lies below or on the straight segment joining any two points on its graph over a convex domain. For convex objectives, every local minimum is global. Neural-network objectives are generally nonconvex; a low training loss does not certify a global optimum or good performance on new data.

Update method	Data used for a gradient	Trade-off
Full-batch gradient descent	Entire training set	Exact gradient of that dataset's average loss; expensive per update
Stochastic gradient descent (SGD)	One sampled example in the strict definition	Cheap, noisy updates
Mini-batch gradient descent	A sampled group of examples	Common compromise; often also called SGD

An **epoch** is one pass through a training set; a **step** is one parameter update. With 1,000 examples, batch size 100, no dropped examples, and one pass, there are 10 updates. Training mini-batches are usually shuffled; dependence, sampling choices, and batch size influence gradient noise.

▼ Go deeper: momentum, Adam, regularization, and stopping

Momentum combines current and past gradients in a running update direction, which can reduce some oscillation and accelerate progress. **Adam** maintains moving averages of gradients and squared gradients, applies bias corrections, and scales updates by coordinate. It still has a learning rate and can fail with poor settings; it does not guarantee a global minimum. Learning-rate **schedules** vary the rate over training.

Regularization adds preferences or constraints to fitting. A common objective is average data loss plus λ times a penalty; λ controls the penalty strength. An L2 penalty proportional to the sum of squared weights favors smaller weights, but its exact coefficient convention determines its gradient. Regularization changes what is being optimized, not merely how quickly it is optimized.

Early stopping chooses when to stop using validation behavior, keeping the test set for final evaluation. Stopping can also involve an update budget or convergence criterion. A tiny gradient alone can reflect flatness or numerical trouble, and decreasing training loss does not prove generalization. We will study optimizer choices and training diagnostics in later chapters.

6. Use integration to accumulate small contributions

Differentiation describes a local rate; **integration** accumulates contributions over an interval. A definite integral is the limit of sums of thin slices. For positive values, it can be read as area under a curve; negative parts contribute signed area.

$$\int_0^3 2t \, dt = [t^2]_0^3 = 3^2 - 0^2 = 9$$

The integral symbol $\int$ means accumulate, 0 and 3 are the limits, and dt names the variable of integration. Here t^2 is an **antiderivative** of $2t$ because its derivative is $2t$. The fundamental theorem of calculus connects these two operations: under suitable continuity conditions, evaluate an antiderivative at the upper endpoint and subtract its value at the lower endpoint.

This gives meaning to Chapter 7's continuous probability density. Suppose a toy duration T is uniformly distributed between 0 and 0.5 seconds. Its density is 2 per second inside that interval and zero outside. Probability between 0.1 and 0.3 seconds is $2 \times (0.3 - 0.1) = 0.4$. Total area is $2 \times 0.5 = 1$. Density height 2 is valid because probability comes from an area, not the height alone.

For a continuous variable with density $p(t)$, a cumulative probability is an integral of p , and an expectation accumulates $t p(t)$. These are the continuous counterparts of adding probabilities and probability-weighted values. Integration also appears when averaging over uncertain quantities; practical methods may use numerical approximation or sampling.

7. Measure surprise and average uncertainty

In information theory, **self-information**, or surprisal, of an outcome with probability p is $-\log_2 p$. A rare outcome carries more information under that distribution. This is a mathematical measure of surprise, not a measure of truth, usefulness, or human understanding.

Probability of the observed outcome	Surprisal
1	0 bits
1/2	1 bit
1/8	3 bits

Using base-2 logarithms gives **bits**; using natural logarithms gives **nats**. The logarithm turns products into sums, so surprise adds for independent outcomes. An outcome assigned probability zero has infinite surprisal if it occurs.

Entropy is the expected surprisal of a distribution. For a toy job outcome with probabilities $P(\text{healthy}) = 0.75$ and $P(\text{failed}) = 0.25$:

$$H(P) = - \sum_i p_i \log_2 p_i = -0.75 \log_2(0.75) - 0.25 \log_2(0.25) \approx 0.8113 \text{ bits}$$

➤ Binary entropy is largest at an even split

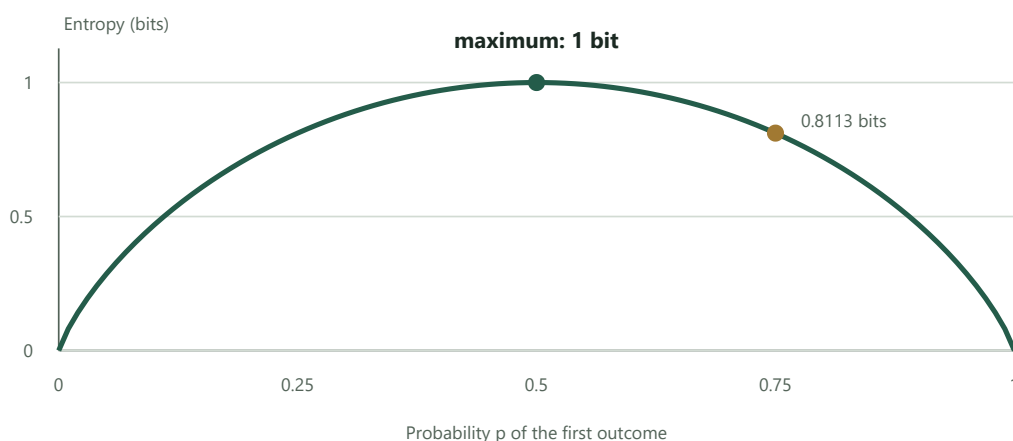


Figure 3. For probabilities $[p, 1 - p]$, entropy is 1 bit at $p = 0.5$ and 0 at either certainty endpoint. At $p = 0.25$ or 0.75 it is about 0.8113 bits.

The sum runs over possible categories. A fair binary outcome has entropy 1 bit; a certain outcome has entropy 0. For k categories, the maximum is $\log_2 k$, achieved by a uniform distribution. We use the limiting convention $0 \log 0 = 0$ in the entropy sum; this does not mean $\log 0$ itself is finite.

Entropy characterizes uncertainty under a distribution. A model can be very certain and wrong: low predicted entropy is not proof that its prediction is reliable.

8. Compare a predicted distribution with a target

Cross-entropy scores predicted probabilities Q using a target distribution P . Keep the same category order. For $P = [0.75, 0.25]$ and the uniform prediction $Q = [0.5, 0.5]$:

$$H(P, Q) = - \sum_i p_i \log_2 q_i = -0.75 \log_2(0.5) - 0.25 \log_2(0.5) = 1 \text{ bit}$$

Entropy uses P both for the weights and inside the logarithm. Cross-entropy keeps P as the weights but uses the prediction Q inside the logarithm. That change is the whole distinction.

Kullback-Leibler (KL) divergence measures the excess cross-entropy over the entropy of P :

$$D_{\text{KL}}(P \| Q) = H(P, Q) - H(P) = 1 - 0.811278 \approx 0.188722 \text{ bits}$$

For valid discrete distributions, KL is nonnegative and zero exactly when they match. It is generally asymmetric and is not a mathematical distance metric. Holding P fixed, minimizing cross-entropy also minimizes KL because $H(P)$ does not depend on Q . See [D2L's information-theory chapter](#) for the coding interpretation and derivations.

In supervised classification, one observed label is commonly represented by a **one-hot target**: 1 for the observed category and 0 for the others. Cross-entropy then reduces to the negative log of the probability assigned to that category. Using natural logs, a true-category probability of 0.8 gives **$-\ln(0.8) \approx 0.2231$ nats**; probability 0.2 gives **1.6094 nats**. Confident mistakes are penalized more strongly.

▼ Go deeper: likelihood, zero probabilities, and mutual information

Under a model that treats labeled examples as conditionally independent given inputs and parameters, the dataset likelihood is a product of assigned label probabilities. Taking its negative log turns that product into a sum of losses. Minimizing this **negative log-likelihood (NLL)** is equivalent to maximum likelihood under those assumptions. Averaging instead of summing changes the scale, not the minimizer for a fixed dataset.

If $p_i > 0$ but $q_i = 0$, cross-entropy and KL are infinite. Terms with $p_i = 0$ contribute zero by convention. Code may use clipping or stable formulas, but clipping changes the exact objective near the boundaries; it is not a mathematical identity.

Conditional entropy $H(Y | X)$ is the average uncertainty about Y after X is known. **Mutual information** $I(X; Y) = H(Y) - H(Y | X)$ measures the reduction in uncertainty for discrete variables with finite entropies. It can detect dependence beyond linear correlation, but dependence alone does not establish causality. These quantities support later topics such as feature selection and representation learning.

9. Connect a probability loss to a parameter update

A binary classifier can turn a real-valued score z , called a **logit**, into a probability with the **sigmoid** $q = 1/(1 + e^{-z})$. Let y be the observed label, 0 or 1. Binary cross-entropy using natural logs is:

$$L = -[y \ln q + (1 - y) \ln(1 - q)], \quad \frac{dL}{dz} = q - y$$

The derivative follows by chaining the loss derivative through the sigmoid: $dL/dq = -y/q + (1 - y)/(1 - q)$, and $dq/dz = q(1 - q)$. Multiplying and simplifying gives $q - y$, for finite z and hence $0 < q < 1$.

If $y = 1$ and $q = 0.8$, $dL/dz = -0.2$. A small descent step on z increases z , which increases q and reduces this example's loss. If $z = wx + b$, apply the chain rule again: $\partial L/\partial w = (q - y)x$. In actual training the parameters w and b are updated, not an independently stored probability for each example.

This is the connection: choose a probability model, define a loss, differentiate it, then use an optimizer. For multiple classes, **softmax** converts a vector of logits into positive probabilities summing to one by exponentiating and normalizing. The classification and neural-network chapters develop these models and their numerically stable implementations.

Reproduce the simple quadratic update with ordinary Python:

```
w = 0.0
learning_rate = 0.5
for step in range(1, 4):
    gradient = w - 3
    w -= learning_rate * gradient
    loss = 0.5 * (w - 3) ** 2
    print(step, w, loss)
# 1 1.5 1.125
# 2 2.25 0.28125
# 3 2.625 0.0703125
```

Extract the ZIP and run `py calculus_lab.py` from that folder using Python 3.10 or newer. Substitute your working `python` or `python3` command if needed. No extra packages are required. The workbook prints gradient steps, a finite-difference check, entropy, cross-entropy, and KL; it writes no files. All examples are synthetic teaching calculations.

10. Practice: calculate the change and explain it

Exercise 1: follow the chain

For $p = 2w + 1$, $r = p - 5$, and $L = r^2/2$, use $w = 2$. What are p , r , L , and dL/dw ? Does a zero gradient at any point of any function guarantee a minimum?

▼ Reveal the calculation

$p = 5$, $r = 0$, and $L = 0$. The derivative is $r \times 1 \times 2 = 0$. This point is a global minimum of this nonnegative squared loss. Zero gradient alone does not guarantee a minimum for other functions; a maximum or saddle can also have zero gradient.

Exercise 2: diagnose a learning rate

For $L(w) = (w - 3)^2/2$, start at $w = 0$ and use $\eta = 2$. Calculate the first two updated weights and their losses. Is crossing the minimum always a failure?

▼ Reveal the update

The first gradient is -3 , giving $w = 6$ and loss 4.5 . The next gradient is 3 , giving $w = 0$ and loss 4.5 again. This rate cycles without improving the objective. Crossing alone is not failure: with $\eta = 1.5$, the iterates cross while their distance from 3 shrinks.

Exercise 3: compare two probability forecasts

Use $P = [0.75, 0.25]$. Compare $Q_1 = [0.75, 0.25]$ with $Q_2 = [0.5, 0.5]$, using bits. Which has lower cross-entropy? What happens if Q assigns zero probability to the second category?

▼ Reveal the comparison

Q_1 matches P , so cross-entropy equals entropy, approximately 0.811278 bits, and KL is zero. Q_2 has cross-entropy 1 bit and KL approximately 0.188722 bits. Giving the second category zero predicted probability makes cross-entropy infinite because P assigns it positive probability.

11. Check your understanding

Answer all five questions, then read the explanations.

01 A differentiable loss has derivative -3 at the current weight. What does a small gradient-descent update do?

- A. Decrease the weight because the derivative is negative.
- B. Increase the weight by subtracting a negative quantity.
- C. Replace the weight with -3 .

Answer: B. The update is $w - \eta g$ with η positive. A negative g produces a positive change in w . The local decrease still depends on taking a suitable step.

02 What does backpropagation compute?

- A. Gradients through a computation using the chain rule.
- B. A guarantee that the global minimum has been reached.
- C. The labels for unseen examples.

Answer: A. Backpropagation propagates sensitivities backward to compute gradients. The optimizer is the separate part that uses those gradients to update parameters.

03 For $L(w) = (w - 3)^2/2$, start at 0 and use learning rate 2. What happens?

- A. The loss falls at every step.
- B. The weight reaches 3 in one update.
- C. The weight cycles between 0 and 6 with unchanged loss.

Answer: C. The error $w - 3$ is multiplied by $1 - \eta = -1$ each step, reversing sign while keeping its magnitude. These rates are specific to this quadratic.

04 For $P = [0.75, 0.25]$ and $Q = [0.5, 0.5]$, which statement is correct in bits?

- A. Entropy and cross-entropy are both zero.
- B. Cross-entropy is 1; KL is about 0.188722.
- C. KL is negative because the prediction is uniform.

Answer: B. $H(P)$ is about 0.811278, $H(P, Q)$ is 1, and KL is their difference. KL is nonnegative for valid discrete probability distributions.

05 A continuous density equals 2 per second from 0 to 0.5 seconds. What is $P(0.1 \leq T \leq 0.3)$?

- A. 2, because density is probability.
- B. 0.2, because only interval width matters.
- C. 0.4, from density times interval width.

Answer: C. Integrate density over the interval: $2 \times (0.3 - 0.1) = 0.4$. Density can exceed 1; total probability is the area, which here is $2 \times 0.5 = 1$.

12. Connect the objective to the learning process

Name the quantity being optimized, identify its parameters, calculate the right gradient, and choose how to update. Then evaluate on appropriate held-out data. Better optimization of a chosen objective is only useful when that objective and evaluation reflect the task you care about.

▼ Quick-reference glossary

Term	Recall cue
Derivative / partial derivative	Local rate / vary one input while holding others fixed
Gradient / Hessian	First-derivative vector / second-derivative matrix
Chain rule / backpropagation	Combine local sensitivities / propagate them backward through a graph
Learning rate / step	Update scale / one parameter update
Epoch / mini-batch	One pass through data / examples used together for an update
Stationary point / convex objective	Zero gradient / every local minimum is global
Integral / antiderivative	Accumulation / function whose derivative is the integrand
Surprisal / entropy	Information of one outcome / its expected value under a distribution
Cross-entropy / KL	Score a forecast under a target / excess over target entropy
Bit / nat	Information units from base-2 / natural logarithms
Logit / sigmoid	Real-valued binary score / map that score to a probability

Examples and diagrams are original; data, probabilities, and losses are chosen for learning. The optimizer performs actual arithmetic on the stated function. Optional topics are foundations for later ML and neural-network chapters, not claims of having covered every advanced method.

Next: Chapter 9: Prompting and context design begins the application-building part with instructions, evidence, and output contracts.